

Product Management System

Add Product

Product List

No products found.

Update Product

Delete Product

Add Product

Product List

Refresh Products

ID: 2

Name: phone

Description: vivo

Price: ₹20000

Quantity: 5

ID: 3

Name: ASUS

Description: GAMING LAPTOP

Price: ₹60000

Quantity: 2

Update Product

2

POCO

Gaminig Phone

25000

2

Update Product

ID: 2
Name: POCO
Description: Gaminig Phone
Price: ₹25000
Quantity: 2

ID: 3
Name: ASUS
Description: GAMING LAPTOP
Price: ₹60000
Quantity: 2

Product List

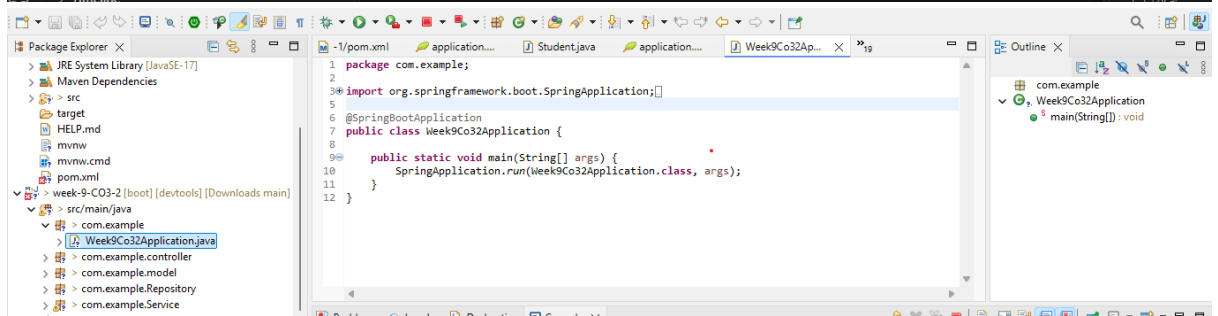
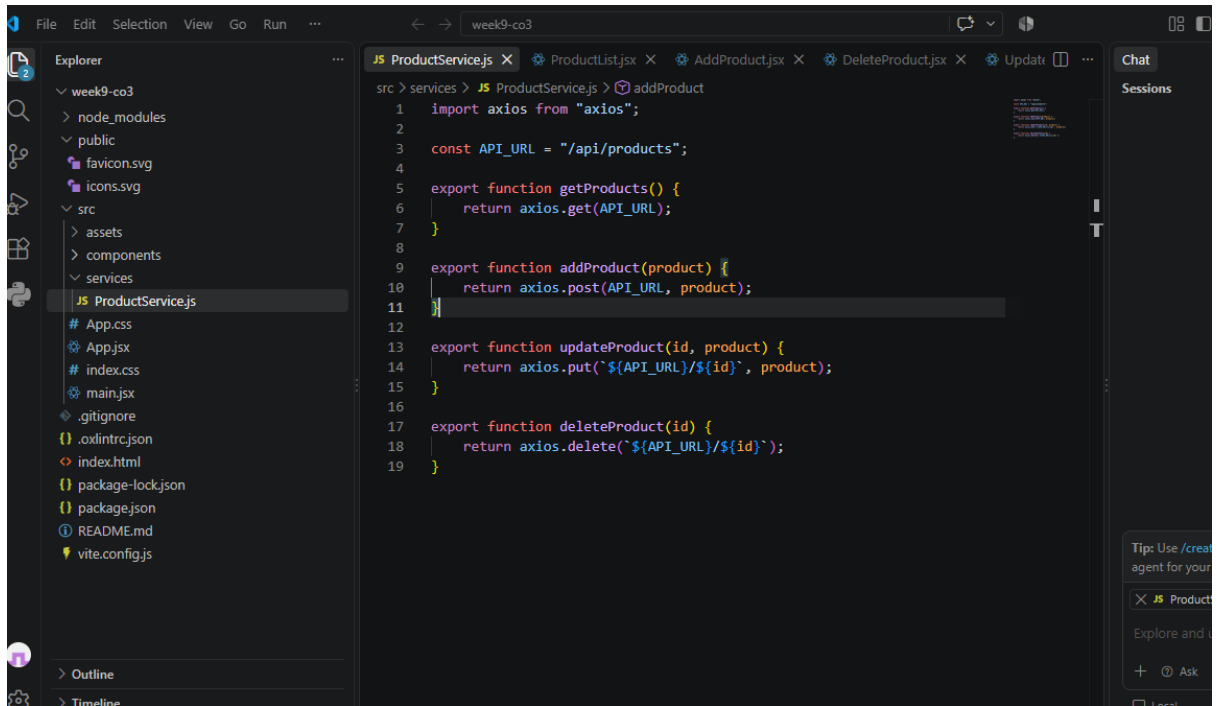
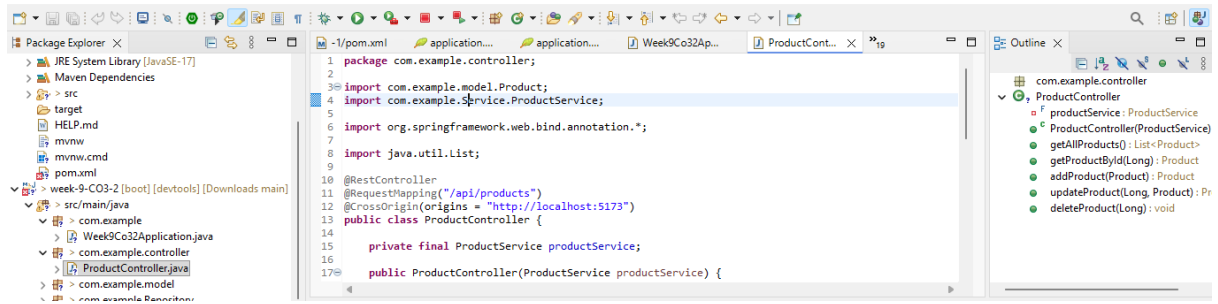
Refresh Products

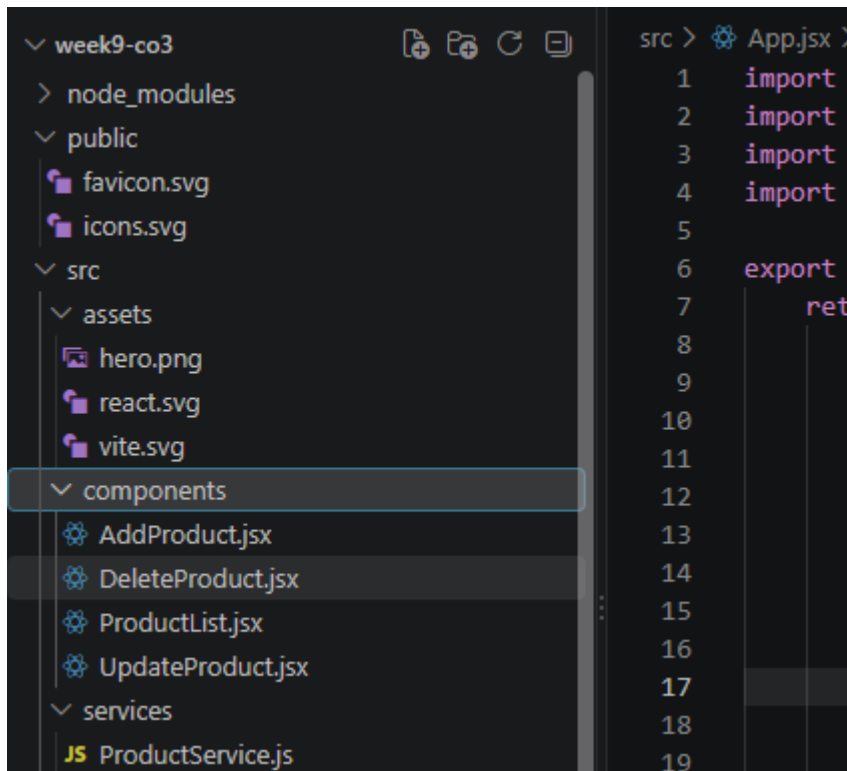
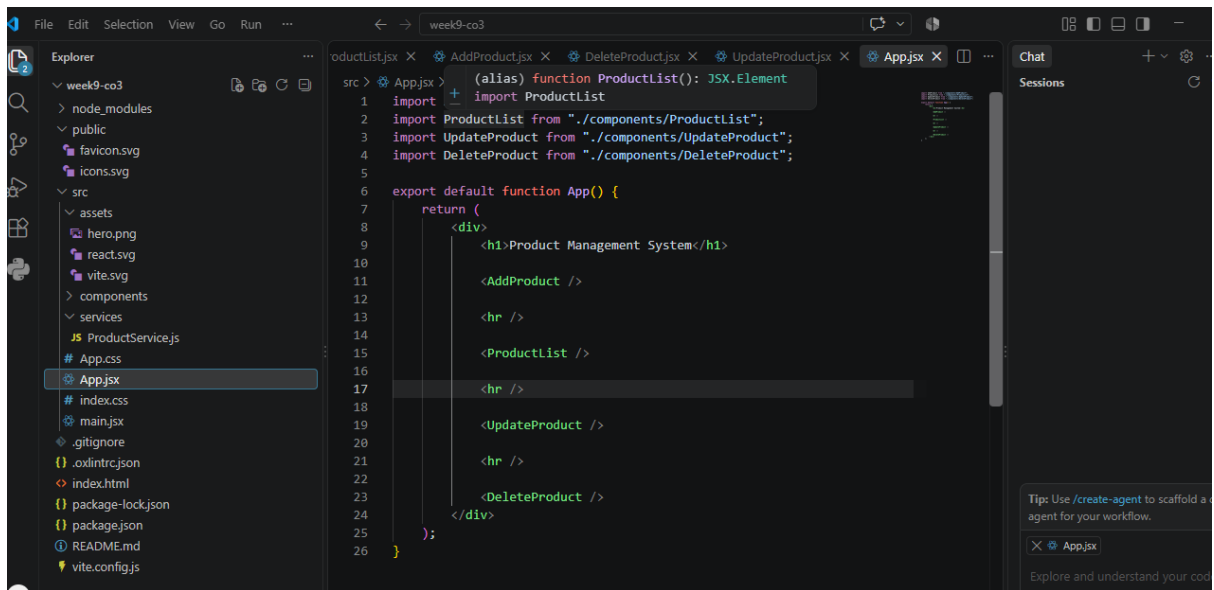
ID: 3
Name: ASUS
Description: GAMING LAPTOP
Price: ₹60000
Quantity: 2

```
1 spring.application.name=week-9-C03-2
2 spring.datasource.url=jdbc:mysql://localhost:3306/week9_db
3 spring.datasource.username=root
4 spring.datasource.password=Charan123
5 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
6
7 spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
8 spring.jpa.hibernate.ddl-auto=update
9 spring.jpa.show-sql=true
10
11 server.port=8080
```

2026-09-19T08:31:14.007+05:30 INFO 11268 --- [week-9-C03-2] [restartedMain] com.example.Week9C032Application : Started Week9C032App1
2026-09-19T08:31:25.600+05:30 INFO 11268 --- [week-9-C03-2] [nio-8080-exec-1] o.s.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring D
2026-09-19T08:31:25.600+05:30 INFO 11268 --- [week-9-C03-2] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet
2026-09-19T08:31:25.600+05:30 INFO 11268 --- [week-9-C03-2] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initializat

Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0
Hibernate: insert into products (description,name,price,quantity) values (?,?,?,?)
Hibernate: select count(*) from products p1_0 where p1_0.id=?
Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0 where p1_0.id=?
Hibernate: update products set description=?,name=?,price=?,quantity=? where id=?
Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0
Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0 where p1_0.id=?
Hibernate: delete from products where id=?
Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0
Hibernate: select p1_0.id,p1_0.description,p1_0.name,p1_0.price,p1_0.quantity from products p1_0





```
import { useState } from "react";
import { addProduct } from "../services/ProductService";

export default function AddProduct() {

  const [name, setName] = useState("");
  const [description, setDescription] = useState("");
  const [price, setPrice] = useState("");
  const [quantity, setQuantity] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();

    const product = {
      name: name,
      description: description,
      price: Number(price),
      quantity: Number(quantity)
    };

    try {
      await addProduct(product);

      alert("Product added successfully!");

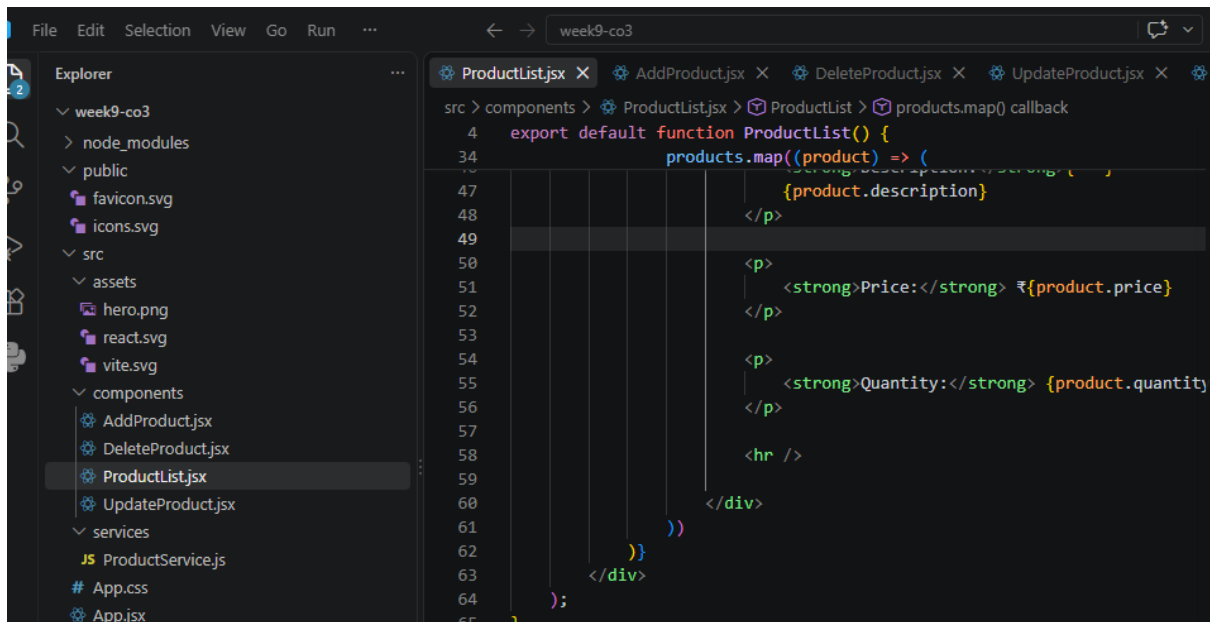
      setName("");
      setDescription("");
      setPrice("");
      setQuantity("");
    } catch (error) {
```

```
1 import { useState } from "react";
2 import { deleteProduct } from "../services/ProductService";
3
4 export default function DeleteProduct() {
5
6     const [id, setId] = useState("");
7
8     const handleSubmit = async (e) => {
9         e.preventDefault();
10
11         try {
12             await deleteProduct(id);
13
14             alert("Product deleted successfully!");
15             function alert(message?: any): void
16                 window.alert() instructs the browser to display a dialog with an optional message, and to wait until
17                 user dismisses the dialog.
18                 MDN Reference
19             alert("Failed to delete product");
20         } catch (error) {
21             console.error(error);
22         }
23     };
24
25     return (
26         <div>
27             <h2>Delete Product</h2>
28
29             <form onSubmit={handleSubmit}>
30
31                 <input
32                     type="number"
33                     placeholder="Product ID"
34                 />
35             </form>
36         </div>
37     );
38 }
```

```
1 import { useState } from "react";
2 import { addProduct } from "../services/ProductService";
3
4 export default function AddProduct() {
5
6     const [name, setName] = useState("");
7     const [description, setDescription] = useState("");
8     const [price, setPrice] = useState("");
9     const [quantity, setQuantity] = useState("");
10
11     const handleSubmit = async (e) => {
12         e.preventDefault();
13
14         try {
15             await addProduct(product);
16
17             alert("Product added successfully!");
18
19             setName("");
20             setDescription("");
21             setPrice("");
22             setQuantity("");
23         } catch (error) {
24             console.error(error);
25             alert("Failed to add product");
26         }
27     };
28
29     return (
30         <div>
31             <h2>Add Product</h2>
32
33             <form onSubmit={handleSubmit}>
34
35                 <input
36                     type="text"
37                     placeholder="Product Name"
38                     value={name}
39                     onChange={(e) => setName(e.target.value)}
40                     required
41                 />
42
43                 <input
44                     type="text"
45                     placeholder="Product Description"
46                     value={description}
47                     required
48                 />
49
50                 <input
51                     type="text"
52                     placeholder="Product Price"
53                     value={price}
54                     required
55                 />
56
57                 <input
58                     type="text"
59                     placeholder="Product Quantity"
60                     value={quantity}
61                     required
62                 />
63
64                 <button type="submit">Add Product</button>
65             </form>
66         </div>
67     );
68 }
```

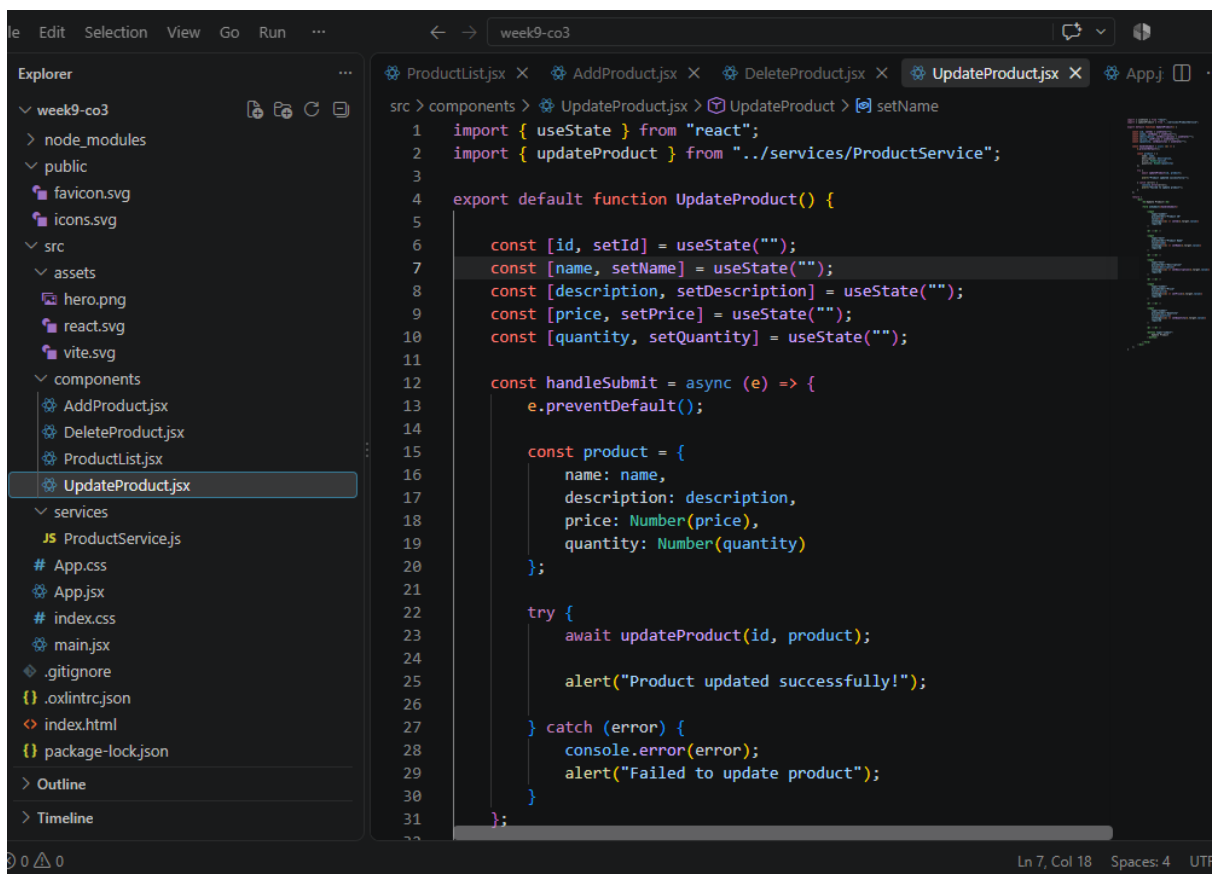
```
src > components > ProductList > products.map() callback
1  import { useEffect, useState } from "react";
2  import { getProducts } from "../services/ProductService";
3
4  export default function ProductList() {
5
6      const [products, setProducts] = useState([]);
7
8      const loadProducts = async () => {
9          try {
10             const response = await getProducts();
11             setProducts(response.data);
12          } catch (error) {
13             console.error(error);
14          }
15      };
16
17      useEffect(() => {
18          loadProducts();
19      }, []);
20
21      return (
22          <div>
23              <h2>Product List</h2>
24
25              <button onClick={loadProducts}>
26                  Refresh Products
27              </button>
28
29              <br /><br />
30
31              {products.length === 0 ? (
```

```
32              <p>No products found.</p>
33          ) : (
34              products.map((product) => (
35                  <div key={product.id}>
36
37                      <p>
38                          <strong>ID:</strong> {product.id}
39                      </p>
40
41                      <p>
42                          <strong>Name:</strong> {product.name}
43                      </p>
44
45                      <p>
46                          <strong>Description:</strong>{" "}
47                          {product.description}
48                      </p>
```



This screenshot shows the VS Code editor with the file explorer on the left and the code editor on the right. The file explorer shows the project structure for 'week9-co3', including 'node_modules', 'public', 'src', and 'services'. The 'src' folder contains 'components', 'assets', and 'services'. The 'components' folder contains 'AddProduct.jsx', 'DeleteProduct.jsx', 'ProductList.jsx', and 'UpdateProduct.jsx'. The 'services' folder contains 'ProductService.js'. The code editor shows the 'ProductList.jsx' file, which is a React component that uses the 'useState' hook to manage state. The component is named 'ProductList' and is exported as the default export. It uses the 'products.map()' method to render a list of products. The code is as follows:

```
src > components > ProductList.jsx > ProductList > products.map() callback
4 export default function ProductList() {
34   products.map((product) => (
47     {product.description}
48   )
49 )
50   <p>
51     <strong>Price:</strong> ₹{product.price}
52   </p>
53   <p>
54     <strong>Quantity:</strong> {product.quantity}
55   </p>
56   <hr />
57 )
58 }
59 </div>
60 </div>
61 </div>
62 </div>
63 </div>
64 </div>
65 </div>
```



This screenshot shows the VS Code editor with the file explorer on the left and the code editor on the right. The file explorer shows the project structure for 'week9-co3', including 'node_modules', 'public', 'src', and 'services'. The 'src' folder contains 'components', 'assets', and 'services'. The 'components' folder contains 'AddProduct.jsx', 'DeleteProduct.jsx', 'ProductList.jsx', and 'UpdateProduct.jsx'. The 'services' folder contains 'ProductService.js'. The code editor shows the 'UpdateProduct.jsx' file, which is a React component that uses the 'useState' hook to manage state. The component is named 'UpdateProduct' and is exported as the default export. It uses the 'useState' hook to manage state for 'id', 'name', 'description', 'price', and 'quantity'. The code is as follows:

```
src > components > UpdateProduct.jsx > UpdateProduct > setName
1 import { useState } from "react";
2 import { updateProduct } from "../services/ProductService";
3
4 export default function UpdateProduct() {
5
6   const [id, setId] = useState("");
7   const [name, setName] = useState("");
8   const [description, setDescription] = useState("");
9   const [price, setPrice] = useState("");
10  const [quantity, setQuantity] = useState("");
11
12  const handleSubmit = async (e) => {
13    e.preventDefault();
14
15    const product = {
16      name: name,
17      description: description,
18      price: Number(price),
19      quantity: Number(quantity)
20    };
21
22    try {
23      await updateProduct(id, product);
24
25      alert("Product updated successfully!");
26
27    } catch (error) {
28      console.error(error);
29      alert("Failed to update product");
30    }
31  };
32 }
```

```
src > components > UpdateProduct.jsx > UpdateProduct > setName
4 export default function UpdateProduct() {
11
12     const handleSubmit = async (e) => {
13         e.preventDefault();
14
15         const product = {
16             name: name,
17             description: description,
18             price: Number(price),
19             quantity: Number(quantity)
20         };
21
22         try {
23             await updateProduct(id, product);
24
25             alert("Product updated successfully!");
26
27         } catch (error) {
28             console.error(error);
29             alert("Failed to update product");
30         }
31     };
32
33     return (
34         <div>
35             <h2>Update Product</h2>
36
37             <form onSubmit={handleSubmit}>
38
39                 <input
40                     type="number"
```

```
src > components > UpdateProduct.jsx > UpdateProduct > setName
4 export default function UpdateProduct() {
31     };
32
33     return (
34         <div>
35             <h2>Update Product</h2>
36
37             <form onSubmit={handleSubmit}>
38
39                 <input
40                     type="number"
41                     placeholder="Product ID"
42                     value={id}
43                     onChange={(e) => setId(e.target.value)}
44                     required
45                 />
46
47                 <br /><br />
48
49                 <input
50                     type="text"
51                     placeholder="Product Name"
52                     value={name}
53                     onChange={(e) => setName(e.target.value)}
54                     required
55                 />
56
57                 <br /><br />
58
59                 <input
60                     type="text"
```

Spec Hub is your single home for API design and governance. Author specs visually, convert them to collections that stay in sync, and enforce standards across your team with custom governance rules.

Postman

> .postman

> postman

http://localhost:8080/api/products

GET

http://localhost:8080/api/products

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

1 {

2 "name": "Gaming Laptop",

3 "description": "ASUS Gaming Laptop",

4 "price": 75000,

5 "quantity": 3

6 }

Body

Cookies

Headers 6

Test Results

200 OK

52 ms

316 B

...

{ } JSON

Preview

Visualize

1 [

2 {

3 "description": "GAMING LAPTOP",

4 "id": 3,

5 "name": "ASUS",

6 "price": 60000.0,

7 "quantity": 2

8 }

9]

New Chat

Describe what you want to do, / for Skills

Auto